ARRAYS, STRINGS & HASHING

FAANG Master Guide

>

🎬 THE REAL-WORLD STORY

Imagine searching for a contact in a phonebook with 1,000,000 contacts. Scanning entry by entry takes 1,000,000 checks in the worst case!

💡 THE HASHMAP AHA MOMENT

What if every contact name instantly transformed into an exact page number index via a mathematical hash function? Lookup time drops from **O(N) to O(1)**!

🤔 WHY DO ARRAYS ALONE FAIL?

- **Array Lookup by Index:** Fast O(1) if index `arr[i]` is known.
- **Array Search by Value:** Slow O(N) linear scan required.
- **Array Insertion at Start:** Slow O(N) shift of all elements!
- **HashMap Solution:** Sacrifices space to map `Value → Index` for instantaneous O(1) lookups!

🛠 HASH FUNCTION MECHANICS

Key → Hash Code → Bucket Index:

```
Key: "alice"
  ↓ Hash Function: hash("alice") = 153029
Bucket Index = 153029 % array_size (16) = 5
  ↓ Direct Array Access!
Bucket[5] stores ("alice", phoneNumber)
O(1) Instant Lookup!
```

📚 PREREQUISITES & COMPLEXITY REASONING

Operation / Structure	Time Complexity	Auxiliary Space
Array Index Access (`arr[i]`)	**O(1)**	**O(N) contiguous memory**
Linear Search / Scan	**O(N)**	**O(1) space**
HashMap Lookup / Put	**O(1) average**	**O(N) space**
Prefix Sum Array Query	**O(1) per range query**	**O(N) prefix array**

🔥 CLUES THAT SCREAM HASHING

1. *"Find a pair that sums to Target"* → HashMap storing `(Target - num) -> Index`
2. *"Check for duplicates or count frequencies"* → HashSet or Frequency Array `int[26]`
3. *"Group anagrams together"* → HashMap storing `SortedString → List`

>

1 4 ESSENTIAL JAVA HASHING SNIPPETS

```
// 1. Frequency Map Increment:
map.put(num, map.getOrDefault(num, 0) + 1);

// 2. Character Frequency Array ('a'-'z'):
int[] freq = new int[26];
for (char c : str.toCharArray()) freq[c - 'a']++;

// 3. Convert Char Array to String Key:
String key = new String(freqChars);
```

2 PREFIX SUM RANGE QUERY UTILITY

```
class PrefixSumUtility {
  private int[] prefix;
  public PrefixSumUtility(int[] nums) {
    prefix = new int[nums.length + 1];
    for (int i = 0; i < nums.length; i++) prefix[i + 1] = prefix[i] + nums[i];
  }
  public int query(int L, int R) { return prefix[R + 1] - prefix[L]; }
}
```

PATTERN 1 COMPLEMENT HASHING (Two Sum) e.g. Two Sum (LC 1)

WHEN TO USE

Finding two numbers in an unsorted array that add up to a target sum in a single pass O(N) time.

TEMPLATE — LC 1

```
public int[] twoSum(int[] nums, int target) {
  Map<Integer, Integer> map = new HashMap<>();
  for (int i = 0; i < nums.length; i++) {
    int complement = target - nums[i];
    if (map.containsKey(complement)) return new int[]{map.get(complement), i};
    map.put(nums[i], i);
  }
  return new int[0];
}
```

PATTERN 2 HASHSET DUPLICATE DETECTION (Contains Duplicate) e.g. Contains Duplicate (LC 217)

WHEN TO USE

Check if any value appears at least twice in an array in O(N) time and O(N) space.

TEMPLATE — LC 217

```
public boolean containsDuplicate(int[] nums) {
  Set<Integer> set = new HashSet<>();
```

>

PATTERN 3 CHARACTER FREQUENCY COUNTING (Valid Anagram) e.g. Valid Anagram (LC 242)

WHEN TO USE

Check if string t is an anagram of s in O(N) time and O(1) space using fixed `int[26]` frequency array.

TEMPLATE — LC 242

```
public boolean isAnagram(String s, String t) {
  if (s.length() ≠ t.length()) return false;
  int[] count = new int[26];
  for (int i = 0; i < s.length(); i++) {
    count[s.charAt(i) - 'a']++; count[t.charAt(i) - 'a']--;
  }
  for (int c : count) if (c ≠ 0) return false;
  return true;
}
```

PATTERN 4 HASHMAP GROUPING (Group Anagrams) e.g. Group Anagrams (LC 49)

WHEN TO USE

Group strings with identical character counts using canonical sorted string key.

TEMPLATE — LC 49

```
public List<List<String>> groupAnagrams(String[] strs) {
  Map<String, List<String>> map = new HashMap<>();
  for (String s : strs) {
    char[] ca = s.toCharArray(); Arrays.sort(ca);
    String key = String.valueOf(ca);
    map.computeIfAbsent(key, k → new ArrayList<>()).add(s);
  }
  return new ArrayList<>(map.values());
}
```

PATTERN 5 BUCKET SORT FREQUENCY AGGREGATION e.g. Top K Frequent Elements (LC 347)

WHEN TO USE

Find top K most frequent elements in O(N) time using frequency buckets instead of heap sorting O(N log K).

TEMPLATE — LC 347

```
public int[] topKFrequent(int[] nums, int k) {
  Map<Integer, Integer> count = new HashMap<>();
  for (int num : nums) count.put(num, count.getOrDefault(num, 0) + 1);
  List<Integer>[] bucket = new List[nums.length + 1];
  for (int key : count.keySet()) {
    int freq = count.get(key);
    if (bucket[freq] == null) bucket[freq] = new ArrayList<>();
    bucket[freq].add(key);
  }
  int[] res = new int[k]; int idx = 0;
  for (int i = bucket.length - 1; i ⩾ 0 && idx < k; i--) {
    if (bucket[i] ≠ null) for (int num : bucket[i]) res[idx++] = num;
  }
  return res;
}
```

PATTERN 6 PREFIX & SUFFIX PRODUCT ACCUMULATION e.g. Product of Array Except Self (LC 238)

WHEN TO USE

Calculate prefix products left-to-right, then multiply by suffix products right-to-left in O(N) time without division!

TEMPLATE — LC 238

```
public int[] productExceptSelf(int[] nums) {
  int n = nums.length; int[] res = new int[n]; res[0] = 1;
  for (int i = 1; i < n; i++) res[i] = res[i - 1] * nums[i - 1];
  int R = 1;
  for (int i = n - 1; i ⩾ 0; i--) { res[i] *= R; R *= nums[i]; }
  return res;
}
```

>

PATTERN 7 HASHSET SEQUENCE START IDENTIFICATION e.g. Longest Consecutive Sequence (LC 128)

SEQUENCE START CONDITION

TEMPLATE — LC 128

>

EASY PROBLEMS

LC 1 · Two Sum — EASY

Map `target - num` to index.

```
public int[] twoSum(int[] nums, int target) {
  Map<Integer, Integer> map = new HashMap<>();
  for (int i = 0; i < nums.length; i++) {
    if (map.containsKey(target - nums[i])) return new int[]{map.get(target - nums[i]), i};
    map.put(nums[i], i);
  }
  return new int[0];
}
```

LC 217 · Contains Duplicate — EASY

`set.add(num)` returns false if present.

```
public boolean containsDuplicate(int[] nums) {
  Set<Integer> set = new HashSet<>();
  for (int n : nums) if (!set.add(n)) return true;
  return false;
}
```

LC 242 · Valid Anagram — EASY

Fixed `int[26]` frequency matching.

```
public boolean isAnagram(String s, String t) {
  if (s.length() ≠ t.length()) return false;
  int[] c = new int[26];
  for (int i = 0; i < s.length(); i++) { c[s.charAt(i) - 'a']++; c[t.charAt(i) - 'a']--; }
  for (int v : c) if (v ≠ 0) return false;
  return true;
}
```

MEDIUM PROBLEMS

LC 49 · Group Anagrams — MEDIUM

Map sorted string key to list.

```
public List<List<String>> groupAnagrams(String[] strs) {
  Map<String, List<String>> map = new HashMap<>();
  for (String s : strs) {
    char[] ca = s.toCharArray(); Arrays.sort(ca);
    String key = String.valueOf(ca);
    map.computeIfAbsent(key, k → new ArrayList<>()).add(s);
  }
  return new ArrayList<>(map.values());
}
```

LC 347 · Top K Frequent Elements — MEDIUM

Bucket sort by frequency array.

```
public int[] topKFrequent(int[] nums, int k) {
  Map<Integer, Integer> cnt = new HashMap<>();
  for (int n : nums) cnt.put(n, cnt.getOrDefault(n, 0) + 1);
  List<Integer>[] b = new List[nums.length + 1];
  for (int key : cnt.keySet()) {
    int f = cnt.get(key); if (b[f] == null) b[f] = new ArrayList<>(); b[f].add(key);
  }
  int[] res = new int[k]; int idx = 0;
  for (int i = b.length - 1; i ⩾ 0 && idx < k; i--) if (b[i] ≠ null) for (int n : b[i]) res[idx++] = n;
  return res;
}
```

LC 128 · Longest Consecutive — MEDIUM

Count sequence if `!set.contains(n-1)`.

```
public int longestConsecutive(int[] nums) {
  Set<Integer> set = new HashSet<>(); for (int n : nums) set.add(n);
  int maxL = 0;
  for (int n : set) {
    if (!set.contains(n - 1)) {
      int curr = n, len = 1; while (set.contains(curr + 1)) { curr++; len++; }
      maxL = Math.max(maxL, len);
    }
  }
  return maxL;
}
```

COMPLETE ARRAYS, STRINGS & HASHING PROBLEM LADDER SUMMARY

LC #	Problem	Pattern	Time	Space	Diff